

Package ‘bloomjoin’

August 2, 2026

Title Efficient Data Frame Joins Using Bloom Filters

Version 1.0.0

Author Gaurav Sood [aut, cre]

Maintainer Gaurav Sood <gsood07@gmail.com>

Description Provides high-performance join operations for large data frames using Bloom filters to reduce memory usage and improve performance. The package is particularly effective when joining a large data frame with a smaller lookup table, and when many keys in the larger data frame don't have matches. Features include optimized C++ implementation, intelligent join strategy selection, comprehensive NA handling, and support for all standard join types.

URL <https://gojiplus.github.io/bloomjoin/>,
<https://github.com/gojiplus/bloomjoin>

BugReports <https://github.com/gojiplus/bloomjoin/issues>

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Imports dplyr, Rcpp (>= 1.0.0)

LinkingTo Rcpp

Suggests testthat (>= 3.0.0), tibble, bit64, rmarkdown, knitr (>= 1.11), microbenchmark, bench, ggplot2, waldo

Config/testthat/edition 3

VignetteBuilder knitr

Config/roxygen2/version 8.0.0

NeedsCompilation yes

Contents

bloomjoin	2
bloom_join	3
bloom_params	4
hash_keys32	5
Index	6

bloomjoin

BloomJoin: Efficient data.frame joins using probabilistic prefilters

Description

`bloom_join()` wraps the standard dplyr join verbs with a probabilistic pre-filter stage implemented in C++ via Rcpp. The filter trims the rows that reach the expensive join when the overlap between the two tables is small, yielding faster joins without changing the final result.

Details

The exported surface mirrors the dplyr join verbs while adding controls for the pre-filter:

- `engine` selects the probabilistic data structure (currently only Bloom filters are implemented).
- `prefilter_side` decides which input will be filtered before calling the underlying join.
- `fpr` configures the target false positive rate for the Bloom filter.
- `n_hint` allows callers to pass optional distinct-count hints for the join keys which helps size the filter without extra scans.

The function automatically samples the key columns to estimate distinct counts, chooses which side to pre-filter when `prefilter_side = "auto"`, and falls back to the raw dplyr join when the Bloom filter would not remove enough rows to pay for its construction.

Author(s)

Maintainer: Gaurav Sood <gsood07@gmail.com>

Authors:

- Gaurav Sood <gsood07@gmail.com>

See Also

Useful links:

- <https://gojiplus.github.io/bloomjoin/>
- <https://github.com/gojiplus/bloomjoin>
- Report bugs at <https://github.com/gojiplus/bloomjoin/issues>

bloom_join	<i>Join two data frames using a Bloom filter pre-filter</i>
------------	---

Description

Join two data frames using a Bloom filter pre-filter

Usage

```
bloom_join(
  x,
  y,
  by = NULL,
  type = c("inner", "left", "right", "full", "semi", "anti"),
  engine = c("auto", "bloom", "fuse"),
  prefilter_side = c("auto", "x", "y"),
  fpr = 0.01,
  n_hint = NULL,
  verbose = FALSE
)
```

Arguments

x, y	Data frames to join.
by	A character vector or named character vector specifying the join columns, just like the by argument in the dplyr join verbs.
type	Type of join to perform. One of "inner", "left", "right", "full", "semi", or "anti".
engine	Probabilistic pre-filter to use. Currently only "bloom" is implemented; "auto" resolves to "bloom" and "fuse" is reserved for a future binary-fuse implementation.
prefilter_side	Which input to pre-filter. "x" filters the first table, "y" filters the second table, and "auto" picks a side based on join semantics and distinct-count estimates.
fpr	Target false positive rate for the Bloom filter. Must be between 0 and 1.
n_hint	Optional estimated distinct counts for the join keys. Supply a numeric scalar to use the same hint for both tables or a named list/vector with entries x and/or y.
verbose	Logical flag controlling diagnostic output.

Value

A data frame identical to the corresponding dplyr join with an additional "bloomjoin" class and a bloom_metadata attribute describing the pre-filter decision.

Examples

```
x <- data.frame(id = 1:100000, value_x = rnorm(100000))
y <- data.frame(id = 50001:60000, value_y = rnorm(10000))
bloom_join(x, y, by = "id", verbose = TRUE)
```

bloom_params

*Report the Bloom filter chosen for a target false positive rate***Description**

Given an expected number of distinct keys n and a target false positive rate p , report the filter `bloom_join()` will build: total bits m , number of hash functions k , and the false positive rate that combination achieves.

Usage

```
bloom_params(n, p = 0.01)
```

Arguments

n	Numeric scalar (> 0): expected number of distinct keys to insert. Fractional values are allowed (estimates).
p	Numeric scalar in $(0, 1)$: target false positive rate.

Details

The numbers come from the same C++ routine the filter itself uses, so this cannot describe a filter the package does not build. It used to: it advertised a blocked layout, with a `block_bits` argument and a blocks count, that the filter has never implemented, and reported an m and k the filter did not use.

Sizing is the standard one (Broder & Mitzenmacher), as used by Guava's `BloomFilter.create` and Spark's: $\text{bits_per_key} = -\log(p) / (\log(2)^2)$ $k = \log(2) * (m / n)$ $\text{fpr}(m, n, k) = (1 - \exp(-k * n / m))^k$

m is then rounded up to a power of two, because the filter indexes with a bit mask rather than a modulo, and k is taken from the rounded m . Rounding only ever adds bits, so the achieved rate lands at or under the request.

Value

A list with class "bloom_params" containing:

- `n`: input n
- `p_target`: target p
- `m_bits`: total bits, a power of two
- `bytes`: total bytes
- `bits_per_key`: m_bits / n
- `k`: number of hash functions
- `fpr_est`: the rate that (m, n, k) achieves

Examples

```
bp <- bloom_params(1e6, 1e-2)
bp
```

hash_keys32	<i>Hash composite join keys to 32-bit integers (stable & type-aware)</i>
-------------	--

Description

Produces a deterministic 32-bit integer per row from one or more key columns. The hash respects column types, canonicalizes -0 to $+0$, normalizes strings to UTF-8, and maps all NA/NaN variants to a single value. Factors are hashed by their level strings so that factor/character equality works as expected.

Usage

```
hash_keys32(x, by, normalize_strings = TRUE)
```

Arguments

<code>x</code>	A data.frame or tibble containing the key columns.
<code>by</code>	Character vector of column names forming the key, in order.
<code>normalize_strings</code>	Logical; if TRUE (default) strings are normalized to UTF-8 before hashing.

Details

Prior to hashing, POSIXlt columns are converted to POSIXct. When the optional **bit64** package is installed, integer64 columns are converted to their exact decimal string representations so they hash losslessly. Without **bit64**, integer64 inputs fall back to `as.character()` with the same equality semantics.

Value

An integer vector of length `nrow(x)` containing deterministic 32-bit hashes. Values may be negative.

Examples

```
if (interactive()) {
  df <- data.frame(
    id = c(1L, 2L, NA_integer_),
    ts = as.POSIXct("2020-01-01", tz = "UTC") + 0:2,
    value = c("a", "b", NA_character_),
    stringsAsFactors = FALSE
  )
  hash_keys32(df, by = c("id", "ts", "value"))
}
```

Index

bloom_join, [3](#)
bloom_join(), [4](#)
bloom_params, [4](#)
bloomjoin, [2](#)
bloomjoin-package (bloomjoin), [2](#)
hash_keys32, [5](#)